Demystifying Feature Requests: Leveraging LLMs to Refine Feature Requests in Open-Source Software

Pragyan K C[1] Dr. Mitra Bokaei Hosseini[1]
[1]Computer Science Department, University of Texas at San Antonio

Abstract

Feature requests and enhancement suggestions, typically provided by users in natural language (NL). However, these requests often suffer from defects such as ambiguity and incompleteness, making them challenging to interpret. Traditional validation methods (e.g., interviews and workshops) help clarify such defects but are impractical in decentralized environments like open-source software (OSS), where change requests originate from diverse users on platforms like GitHub. We propose a novel approach leveraging Large Language Models (LLMs) to detect and refine NL defects in feature requests. Our approach automates the identification of ambiguous and incomplete requests and generates clarification questions (CQs) to enhance their usefulness for developers. To evaluate its effectiveness, we apply our method to real-world OSS feature requests and compare its performance against human annotations.

Contribution

(1) We conduct an empirical analysis of feature requests to identify common NL defects, including ambiguity and incompleteness in feature requests and develop automated methods leveraging LLMs to detect and classify NL defects in feature requests
(2) We develop methods to automatically generate clarifying questions (CQs) to address ambiguity and incompleteness while preserving the requester's original intent
(3) We conduct interviews with Signal developers to explore how developers perceive NL defects in feature requests, how they address these defects in practice, and the effects of NL defects on downstream software engineering (SE) tasks

Problem Statement

Feature requests written in Natural Language are prone to defects such as **Ambiguity** and **Incompleteness**. Traditional validation methods help clarify such defects but are impractical in decentralized environments like open-source software (OSS), where change requests originate from diverse users on platforms like GitHub.

Feature Request

Request No: 160

Title: Feature: Group notifications by toot

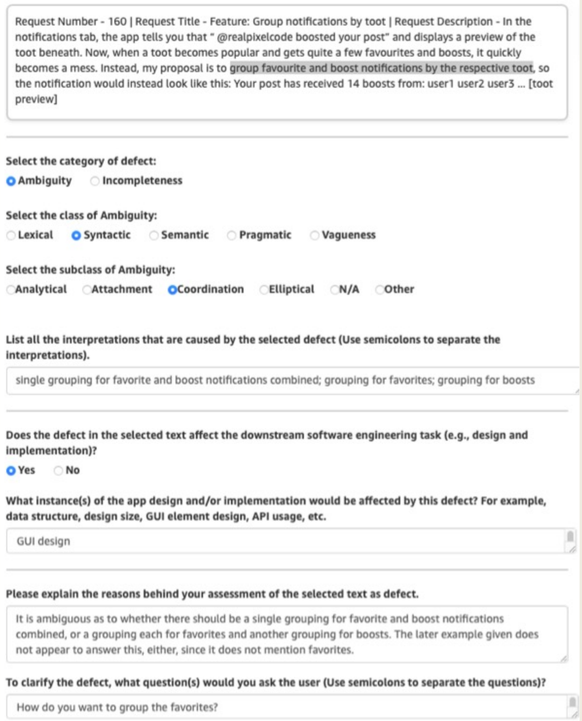
Description: In the notifications tab, the app tells you that "@realpixelcodeboosted your post" and displays a preview of the toot beneath. Now, when a toot becomes popular and gets quite a few favourites and boosts, it quickly becomes a mess.Instead, my proposal is to group favourite and boost notifications by the respective toot, so the notification would instead look like this:Your post has received 14 boosts from:user1user2user3...[toot preview]

How to resolve?

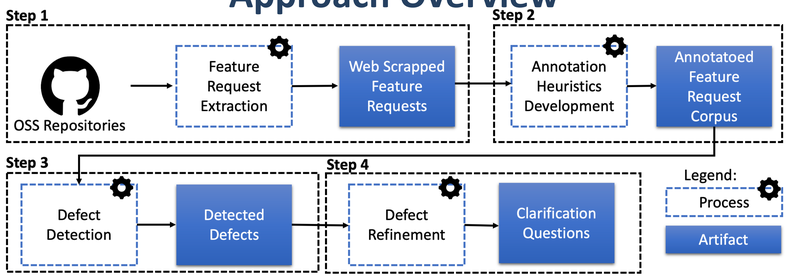
Defect Detection
Detect Natural Language defects: Ambiguity (**Lexical**, **Syntactic**, **Semantic**, **Pragmatic**, **Vagueness**) and **Incompleteness**.

Defect Refinement
Generate **clarifying questions** to **refine ambiguous** or **incomplete feature requests**.

Annotation Tool



Approach Overview



Annotation Format

{"Category": "**Ambiguity**", "Class": "**Syntactic**", "Subclass": "**Coordination**", "Reason": "**It is ambiguous as to whether there should be a single grouping for favorite and boost notifications combined, or a grouping each for favorites and another grouping for boosts. The later example given does not appear to answer this, either, since it does not mention favorites.**", "Questions": "**How do you want to group the favorites?**", "Interpretations": "**single grouping for favorite and boost notifications combined; grouping for favorites; grouping for boosts**", "affectsDownstream": "**Yes**", "affectDescription": "**GUI design**", "text": "**group favourite and boost notifications by the respective toot,**", "start": **357**, "end": **420**}

Dataset Details

	Lexical	Syntactic	Semantic	Pragmatic	Vagueness	Incompleteness
# Requests	24	16	9	17	11	57
# Instances	42	24	10	32	14	57

Experiment Design

Exp 1: Detect Detection	Exp 2: Defect Refinement
Exp 1.1: **Ambiguity Detection without CoT**	Exp 2.1: **Ambiguity Refinement**
Exp 1.2: **Ambiguity Detection with CoT**	Exp 2.2: **Incompleteness Refinement**
Exp 1.3: **Incompleteness Detection without CoT**	

Input – Output for the Model

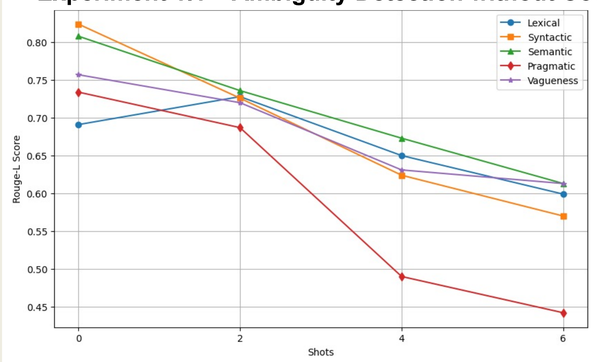


Prompt for Defect Refinement

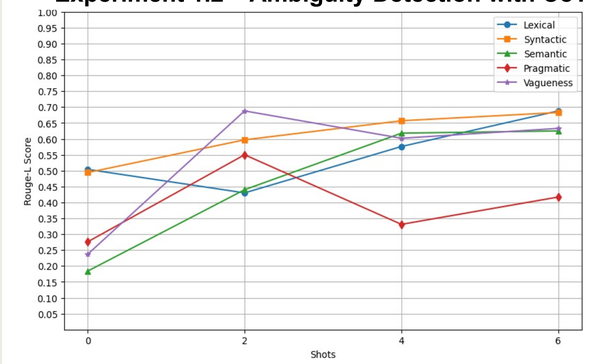
{Ambiguity Sub-Class | Incompleteness}: {Definition} You are a software analyst specializing in {Ambiguity Sub-Class | Incompleteness} resolution in GitHub feature requests.
Carefully read the provided statement along with the {extracted {Ambiguity Sub-Class} segment | missing information} and the reasoning behind why the {extracted {Ambiguity Sub-Class} segment | feature request} is considered {ambiguous | incomplete}. Based on the information, generate clear and concise clarification questions aimed at addressing the {Ambiguity Sub-Class | Incompleteness} in the {extracted {Ambiguity Class} segment | feature request}; include all generated clarifying questions in a single comma-separated list. Make sure to have all elements of the list in quotation marks. Demonstrations are provided for clarity. Each demonstration is separated by the trigger word # END. Inside each demonstration, the statement, reason, {extracted {Ambiguity Sub-Class} segment | missing information}, and clarifying questions are separated using the trigger word ####. Strictly follow the provided demonstration.
<Examples*>
Statement: <Test Feature Request>
Reason: <Test Reason>
{extracted {Ambiguity Sub-Class} segment | missing information}: <Test Text Segment | Test Missing Information>
Clarifying Question(s):

Results

Experiment 1.1 – Ambiguity Detection without CoT



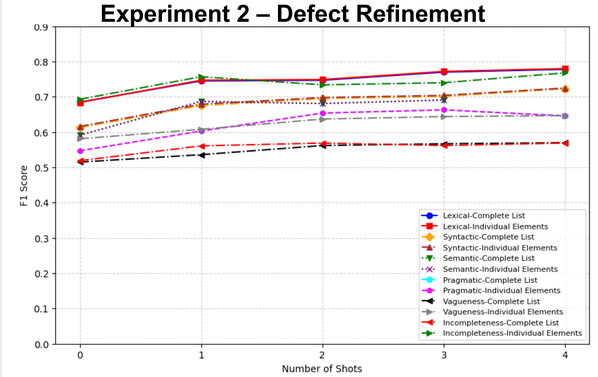
Experiment 1.2 – Ambiguity Detection with CoT



Experiment 1.3 – Incompleteness Detection

# of Shots	0	2	4	6
Precision	0.571	0.571	0.577	0.588
Recall	1.0	1.0	0.997	0.981
F1	0.727	0.727	0.731	0.735

Experiment 2 – Defect Refinement



Conclusion & Future Works

- ✓ Created an annotated corpus of feature requets from OSS
- ✓ Prompted LLM to detect and classify defects in Feature Request
- ✓ Prompted LLM to genearted clarifying questions for detected defects in Feature Requests
- • Alignment with project goals & software requirements